Automated Model Risk Auditor

A test that does not trust a single judge.

That cannot be fooled by good behavior.

And that keeps watching after release.

The features

Part 1: Test suite management

A live, organized test suite where every change creates a new version, and every result is recorded against the version that produced it.

How it works

- A new test is proposed and checked against a fixed structure: which fields, of what type.
- A valid test enters the suite as a new version. Old versions remain.
- When a model is submitted, the system picks one suite version and records it.
- Every result is stored with the suite version and the model version that produced it.

What mistake it prevents

Threat: A score changes between two runs, and there is no way to know whether the model changed or the tests did.

Answer: Every run is tied to a fixed suite version, so results can be reproduced and compared.

Technologies and design decisions

Component	Type
Feature Store pattern (inspired by Feast)	Built from scratch
JSON Schema	Existing technology
PostgreSQL + TimescaleDB	Existing technology
MLflow	Existing technology

Data structure: SuiteVersion

suite_version, test_id, category, created_at, run_id

Part 2: Detecting a model that behaves differently when tested

Every question is asked twice, once as an announced test and once inside a normal conversation, and the answers are compared.

How it works

- Each test has two phrasings: Mode A announces it is a test, and Mode B is a normal conversation with no hint.
- The two answers are converted into numbers that represent meaning, and the distance between them is measured.

- After many questions, a statistical test checks whether the average gap is real or chance.
- A significant gap is a suspicion that the model changes behavior when it knows it is being tested.

What it defends against

Threat: An advanced model notices it is being tested, behaves well, and behaves differently after release.

Answer: Comparing the two modes exposes a gap that cannot be hidden, because it is measured on every question.

Technologies and design decisions

Component	Type
Dual-test design (Mode A / Mode B)	Our decision
Apollo Research method	Existing technology
Sentence-Transformers	Existing technology
SciPy Wilcoxon	Existing technology

Data structure: ModePair

test_id, response_a, response_b, semantic_distance, p_value

Part 3: Three different judges

Every answer is examined by three different methods, and the decision follows the agreement between them.

How it works

- First judge: fixed rules, for example searching for forbidden phrasing.
- Second judge: a model that rates the answer against written criteria.
- Third judge: a model from a different family, rating by the same criteria.
- When all three agree, the decision is automatic. When there is a sharp disagreement, the case goes to a human.

What mistake it prevents

Threat: A single judge errs systematically, because it is blind to the same things the tested model is blind to.

Answer: Judges that differ by nature err in different places, and the agreement measure says when to trust.

Technologies and design decisions

Component	Type
Diverse judge layer	Our decision
Cohen’s Kappa	Existing technology
LangSmith / Weights & Biases	Existing technology

Data structure: JudgeVerdict

response_id, judge, verdict, rationale, kappa_pair

Part 4: Risk score and release gate

All results become one score, but a critical category that crossed a hard threshold fails the model regardless of the score.

How it works

- Each risk category gets its own score.
- The categories are mapped to a standard framework, so the report speaks a familiar language.
- The scores are weighted by severity into an overall score.
- Before the decision, the hard threshold of the critical categories is checked. Crossing a single threshold is enough to fail.

What mistake it prevents

Threat: A high average hides a severe failure in one area, and the model is released.

Answer: A hard threshold for each critical category: severe risk does not average out.

Technologies and design decisions

Component	Type
Hard threshold for critical categories	Our decision
NIST AI RMF	Existing technology
PostgreSQL + TimescaleDB	Existing technology

Data structure: RiskScore

category, score, weight, hard_threshold, gate_result

Part 5: Automatic attack generation

The system invents new attack phrasings with three methods, without exceeding a fixed budget.

How it works

- Start from ready-made phrasings from a vulnerability scanner.
- GCG computes a suffix of words that pushes the model toward answering what is forbidden.
- AutoDAN evolves phrasings: the best survive and mutate.
- PAIR runs a conversation with an attacker model that improves its attempt each round.
- A cost guard counts every call and stops when the ceiling is reached.

What mistake it prevents

Threat: The test is built only from old phrasings, and models have already learned to defend against them.

Answer: Automatic generation of new phrasings, under a ceiling that prevents endless costs.

Technologies and design decisions

Component	Type
PyRIT	Existing technology
Garak	Existing technology
PyTorch autograd	Existing technology
DEAP	Existing technology
Cost Guard	Our decision

Data structure: AttackAttempt

attack_id, method, prompt, cost_usd, outcome

Part 6: Running in an isolated room

Tests that require running code run in an environment that looks real but is sealed off from the world.

How it works

- The system decides how dangerous the test is: light, or one that needs full isolation.
- A dangerous test gets a fresh microVM. A light test gets a lighter protection layer.
- The environment has an empty network: there is no way to reach outside.
- Every attempt to get out is blocked and recorded as a finding. After the test the environment is destroyed.

What it defends against

Threat: The tested model tries, during the test, to touch real systems.

Answer: Full isolation: the attempt is seen and recorded, but cannot do damage.

Technologies and design decisions

Component	Type
Firecracker	Existing technology
gVisor	Existing technology
Network isolation (namespace + iptables)	Existing technology

Data structure: SandboxRun

run_id, isolation_level, network_attempts, started_at, destroyed_at

Part 7: Regression tracking

Every new model version is compared to the previous ones, and only a real change raises an alert.

How it works

- A new model version runs on the same suite version as its predecessor.
- Scores are stored together with the history.

- Each category has a control chart with an expected range, set from the score history.
- A score outside the range is a regression, and an alert is raised before release.

What mistake it prevents

Threat: An update improves one thing and worsens another, and the overall score hides it.

Answer: Control per category, with a range that separates noise from regression.

Technologies and design decisions

Component	Type
SPC (statistical process control)	Existing technology
MLflow	Existing technology
PostgreSQL + TimescaleDB	Existing technology
SciPy Wilcoxon	Existing technology

Data structure: CategoryTrend

category, model_version, score, control_low, control_high

Part 8: A signed report for the regulator

All results are summarized in a three-level report, signed so that any change to it is detected.

How it works

- Every component hands in its results, and these are checked against a fixed structure.
- The report is built in three levels: summary, detail and appendix.
- A PDF is produced from the report.
- The report is digitally signed. Whoever receives it can verify it was not altered.

What mistake it prevents

Threat: A score without explanation, or a report that can be changed after it was produced.

Answer: Three levels for every reader, and any change after signing is detected.

Technologies and design decisions

Component	Type
WeasyPrint	Existing technology
JSON Schema	Existing technology
Ed25519 signature on the report	Existing technology
NIST AI RMF	Existing technology

Data structure: SignedReport

report_id, model_version, verdict, content_hash, signature

Part 9: Monitoring after release

A small sample of real usage passes through the same judges, to detect gradual drift.

How it works

- A small share of real usage of the model is sent to an event stream, without delaying the user.
- The same three judges examine the sample.
- Scores are stored on a timeline, per category.
- Week after week the distribution is compared to the original test. A drift beyond the threshold raises an alert.

What mistake it prevents

Threat: The model degrades gradually after release, and nobody notices until someone complains.

Answer: An eye that stays open, comparing behavior in the field to what was tested.

Technologies and design decisions

Component	Type
Apache Kafka	Existing technology
Evidently AI	Existing technology
PostgreSQL + TimescaleDB	Existing technology
Diverse judge layer	Our decision

Data structure: DriftSample

sample_id, week, category, verdict, drift_score

Six decisions that could have gone the other way

One judge or three? We chose three different judges. Judges that differ by nature are not blind to the same things. When they agree we can trust, and when they disagree a human decides. Price: higher cost, and a decision mechanism to maintain.

Average score only, or also a hard threshold? We chose a hard threshold. Severe risk does not average out. A critical category that crossed the threshold fails the model, regardless of the rest. Price: a model can fail on one finding that looks small, and whoever is waiting for release must be convinced.

Manual scenarios only, or automatic too? We chose automatic generation as well. A real attacker invents. A test that does not invent only checks the past. Price: API call costs, and a budget that must be guarded.

Run directly or inside an isolated room? We chose an isolated microVM. Virtualization-level isolation blocks real damage, while every attempt is still visible. Price: more complex infrastructure, and slightly longer run time.

Once before release or also after? We chose continuous monitoring. Gradual degradation is only caught by monitoring. A small sample is cheap and does not block users. Price: an additional data pipeline, and strict privacy care for the sample.

One score or a signed three-level report? We chose a signed report. Three levels give every reader what they need, and the signature guarantees the report was not altered. Price: the effort of building the report and maintaining a fixed schema.

What the first version taught us

First version	Specification
Ten to twenty hand-built scenarios	A live versioned suite with automatic generation
A rule-based judge only	Three different judges, and agreement-based decisions
We checked only a gap between two modes	A statistical test that separates a real gap from chance
We did not handle models that can act	An isolated execution environment
A single score	A score with a hard threshold, and a signed report